SANDEEP PAL
FULL STACK DEVELOPER (DOTNET,CLOUD,API & REACT)
@Sandeepp_pall

MASTERING DEPENDENCY INJECTION IN .NET

50+ ESSENTIAL Q&A

REAL-WORLD SCENARIOS

MNC INTERVIEW SECRETS

EXPERT INTERVIEW GUIDE FOR .NET DEVELOPERS

(2026 EDITION)

Mastering Dependency Injection in .NET (2026 Edition)

Real MNC Interview Questions + Deep-Dive Answers + Real-World Scenarios

Need Deeper Guidance?
If you want real 1-on-1 help:
Complete Interview Q&A Resources, Live mock interviews,Career mentorship,Resume reviews,LinkedIn profile optimization Connect with me:

https://topmate.io/sandeeppal

https://www.linkedin.com/in/sandeeppal/

PART 1: FUNDAMENTALS + CORE INTERVIEW QUESTIONS (Q1–Q18)

Q1. What is Dependency Injection? Explain in simple terms.

Answer:
Dependency Injection (DI) is a design pattern used to remove tight coupling between classes.
Instead of a class creating its dependencies:

```
public class OrderService
{
  private readonly PaymentService _paymentService = new PaymentService();
}
```

We inject them from outside:

```
public class OrderService
{
  private readonly IPaymentService _paymentService;

  public OrderService(IPaymentService paymentService)
```

```
  {
      _paymentService = paymentService;
  }
}
```

🔍 Real-world analogy:

Think of a car factory:

- Without DI → Car builds its own engine
- With DI → Engine is provided externally

💡 Why companies care:

- Enables unit testing
- Supports clean architecture
- Reduces technical debt

Q2. What problem does DI solve?

Answer:

Problem: Tight Coupling

var service = new EmailService(); // hard dependency
Issues:

- Hard to test
- Hard to replace implementation
- Violates SOLID (Dependency Inversion Principle)

DI solves:

- Loose coupling
- Easier mocking
- Better maintainability

Q3. What are the types of Dependency Injection?

Answer:

1. Constructor Injection (Most Recommended)

public UserController(IUserService userService)

https://www.linkedin.com/in/sandeeppal/

✔ Mandatory dependencies
✔ Immutable
✔ Best practice in .NET

---

2. Property Injection

public IUserService UserService { get; set; }
Optional dependencies
Can cause null issues

---

3. Method Injection

public void Process(IUserService userService)
✔ Used for specific operations only

Interview Insight:

Most MNCs expect:
👉 "Constructor Injection is preferred because it enforces dependency availability and supports immutability."

Q4. What is IoC (Inversion of Control)?

Answer:
IoC means:
👉 "Instead of object controlling dependencies, container controls them."
DI is a way to implement IoC.

🔍 Example:

Without IoC:
var service = new EmailService();
With IoC:
services.AddScoped<IEmailService, EmailService>();
Framework injects automatically.

Q5. What is a DI Container in .NET?

Answer:
A DI container is responsible for:

https://www.linkedin.com/in/sandeeppal/

- Creating objects
- Managing lifetimes
- Injecting dependencies

In .NET:

Built-in container via:
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IService, Service>();

Popular Containers:

- Built-in (.NET Core)
- Autofac
- Ninject
- StructureMap

Q6. Explain Service Lifetimes in .NET

Answer:

1. Transient

services.AddTransient<IService, Service>();

- New instance every time
- Lightweight services

---

2. Scoped

services.AddScoped<IService, Service>();

- One instance per HTTP request

---

3. Singleton

services.AddSingleton<IService, Service>();

- One instance for entire app lifetime

Real-world Example:

Service Type	Example

https://www.linkedin.com/in/sandeeppal/

Transient	Email formatter
Scoped	DbContext
Singleton	Cache service

Q7. What happens if you inject Scoped into Singleton?

Answer:
This is a critical interview question

Problem:

- Singleton lives forever
- Scoped lives per request

👉 Leads to:

- Memory leaks
- Invalid object state

Example:

```
public class SingletonService
{
  public SingletonService(DbContext context) // WRONG
  {
  }
}
```

Fix:

Use IServiceProvider:
var scope = _serviceProvider.CreateScope();

Q8. What is Constructor Injection vs Service Locator?

Answer:

Constructor Injection (Good)

public MyService(IRepo repo)
✔ Explicit dependencies
✔ Testable

---

https://www.linkedin.com/in/sandeeppall/

Service Locator (Anti-pattern)

var repo = serviceProvider.GetService<IRepo>();
Hidden dependencies
Hard to test

Q9. What is Dependency Inversion Principle (DIP)?

Answer:
From SOLID:
👉 High-level modules should not depend on low-level modules.
👉 Both should depend on abstractions.

Example:

public interface ILogger { }
Not:
public class FileLogger { }

Q10. How DI improves Unit Testing?

Answer:
Without DI:
var service = new PaymentService();
With DI:
var mock = new Mock<IPaymentService>();
✔ Easy mocking
✔ Isolated testing

Real Interview Scenario:

"Write unit test for OrderService"
Expected:

- Use Moq
- Inject mock dependency

Q11. What is IServiceCollection?

Answer:
It is a container to register services.
builder.Services.AddScoped<IUserService, UserService>();

https://www.linkedin.com/in/sandeeppall/

Q12. What is IServiceProvider?

Answer:
Used to resolve services:
var service = serviceProvider.GetService<IUserService>();

Q13. What is AddScoped vs AddTransient in real systems?

Answer:

Example:

- Payment processing → Scoped
- Logging formatter → Transient

Q14. Can we inject primitive types using DI?

Answer:
Yes, using Options Pattern
services.Configure<AppSettings>(config);
Inject:
IOptions<AppSettings>

Q15. What is Options Pattern?

Answer:
Used for strongly-typed configuration.

```
{
 "AppSettings": {
   "ApiKey": "123"
 }
}
```

Q16. What is Lazy Injection?

Answer:
Used when dependency is expensive:

https://www.linkedin.com/in/sandeeppal/

Lazy<IService>
✔ Improves performance

Q17. What is Factory Pattern with DI?

Answer:
Used when dynamic object creation is needed.
services.AddTransient<Func<string, IService>>(provider => key => ...);

Q18. What is Open Generic Registration?

Answer:
services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
✔ Used in generic repositories

PART 2: ADVANCED DEPENDENCY INJECTION (Q19–Q40)

Q19. What is Circular Dependency in DI? How do you resolve it?

Answer:
A circular dependency occurs when:
ServiceA → depends on → ServiceB
ServiceB → depends on → ServiceA

Example:

```
public class A
{
  public A(B b) { }
}

public class B
{
  public B(A a) { }
}
```

👉 This will cause runtime failure:

https://www.linkedin.com/in/sandeeppal/

"A circular dependency was detected"

Solutions:

1. Refactor Design (Best Approach)

Break responsibilities using interfaces or mediator pattern.

2. Use Lazy Injection

```
public class B
{
  private readonly Lazy<A> _a;

  public B(Lazy<A> a)
  {
     _a = a;
  }
}
```

3. Use IServiceProvider (Controlled usage)

```
var service = serviceProvider.GetRequiredService<A>();
```

Interview Insight:

MNCs expect:
👉 "Circular dependency indicates bad design, not just a DI problem."

Q20. How DI works internally in .NET?

Answer:
The .NET DI container:

1. Registers services in IServiceCollection
2. Builds a service provider
3. Uses expression trees / compiled delegates for fast object creation
4. Maintains lifetime tracking

🔍 Internal Flow:

Request → Middleware → Controller → Constructor → DI resolves dependencies

---

https://www.linkedin.com/in/sandeeppal/

Important:

- Singleton stored once
- Scoped stored per request
- Transient created every time

Q21. What is IServiceScope and when to use it?

Answer:
Used to create a manual scope.

🔍 Real-world scenario:

Background service needs scoped dependency:

```
using(var scope = _serviceProvider.CreateScope())
{
  var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
}
```

Use Cases:

- Background jobs
- Hosted services
- Console apps

Q22. How DI works in Middleware?

Answer:
Middleware supports constructor injection + method injection

Example:

```
public class MyMiddleware
{
  private readonly RequestDelegate _next;

  public MyMiddleware(RequestDelegate next)
  {
     _next = next;
  }
```

https://www.linkedin.com/in/sandeeppall/

```
  public async Task Invoke(HttpContext context, IMyService service)
  {
      await _next(context);
  }
}
```

Key Point:

- Constructor → singleton dependencies
- Invoke() → scoped dependencies

Q23. What is Dependency Injection in Minimal APIs (.NET 6+)?

Answer:
Dependencies are injected directly into endpoints:

```
app.MapGet("/users", (IUserService userService) =>
{
  return userService.GetAll();
});
```

Benefit:

- Less boilerplate
- Cleaner APIs

Q24. What is Keyed/Named Services in .NET?

Answer:
Used when multiple implementations exist.

Example (.NET 8+):

```
services.AddKeyedScoped<IService, ServiceA>("A");
services.AddKeyedScoped<IService, ServiceB>("B");
```

Inject:

```
public MyClass([FromKeyedServices("A")] IService service)
```

---

https://www.linkedin.com/in/sandeeppal/

Real-world use:

- Payment gateways (Stripe, Razorpay)
- Notification services (Email, SMS)

Q25. What is Scrutor in DI?

Answer:
Library for assembly scanning & decoration

Example:

```
services.Scan(scan => scan
  .FromAssembliesOf(typeof(IService))
  .AddClasses()
  .AsImplementedInterfaces()
  .WithScopedLifetime());
```

Benefit:

- Reduces manual registrations
- Clean architecture

Q26. What is Decorator Pattern with DI?

Answer:
Adds behavior without modifying original class.

Example:

```
public class LoggingService : IService
{
  private readonly IService _inner;

  public LoggingService(IService inner)
  {
     _inner = inner;
  }
}
```

---

https://www.linkedin.com/in/sandeeppal/

Real-world:

- Logging
- Caching
- Retry logic

Q27. What is TryAdd vs Add in DI?

Answer:
services.TryAddScoped<IService, Service>();
✔ Adds only if not already registered

Use Case:

- Libraries/frameworks

Q28. What is Multiple Implementation Injection?

Answer:

```
public class MyService
{
  public MyService(IEnumerable<IService> services)
  {
  }
}
```

Real-world:

- Notification handlers
- Payment processors

Q29. How to implement Factory Pattern using DI?

Answer:

```
public interface IServiceFactory
{
  IService Create(string type);
}
```

---

https://www.linkedin.com/in/sandeeppall/

Implementation:

```
public class ServiceFactory : IServiceFactory
{
  private readonly IServiceProvider _provider;

  public IService Create(string type)
  {
     return type switch
     {
        "A" => _provider.GetService<ServiceA>(),
        _ => throw new Exception()
     };
  }
}
```

Q30. What is IHostedService and DI role?

Answer:
Used for background tasks.

Example:

```
public class Worker : BackgroundService
{
  private readonly IServiceProvider _provider;
}
```

Key:

Use scope inside background service.

Q31. What is ActivatorUtilities?

Answer:
Used to create objects manually with DI support.
var instance = ActivatorUtilities.CreateInstance<MyClass>(provider);

Use Case:

- Dynamic object creation

https://www.linkedin.com/in/sandeeppal/

Q32. How to inject DbContext properly?

Answer:
services.AddDbContext<AppDbContext>();
✔ Scoped by default

Avoid:

- Singleton DbContext

Q33. What is AddDbContextPool?

Answer:
Improves performance by reusing DbContext.
services.AddDbContextPool<AppDbContext>();

Use Case:

High-load applications

Q34. What are common DI performance issues?

Answer:

- Too many transient objects
- Heavy constructors
- Reflection-heavy containers

Optimization:

- Use Singleton wisely
- Avoid deep dependency chains

Q35. What is ValidateScopes in DI?

Answer:
builder.Services.BuildServiceProvider(new ServiceProviderOptions
{

https://www.linkedin.com/in/sandeeppal/

```
  ValidateScopes = true
});
```
✔ Detects lifetime issues early

Q36. What is Auto-wiring in DI?

Answer:
Automatic resolution based on constructor.
✔ Default behavior in .NET

Q37. What is Conditional Dependency Injection?

Answer:
Inject based on condition:

```
if(env.IsDevelopment())
  services.AddScoped<IService, DevService>();
```

Q38. What is Multi-tenant DI?

Answer:
Different tenants use different services.

Example:

- Tenant A → SQL Server
- Tenant B → MongoDB

Approach:

- Custom service provider per tenant

Q39. What is DI in Clean Architecture?

Answer:
- Core layer → interfaces
- Infrastructure → implementations
- API → DI wiring

---

https://www.linkedin.com/in/sandeeppal/

Rule:

👉 Dependencies flow inward

Q40. What are common DI mistakes?

Answer:
Top mistakes:

1. Injecting scoped into singleton
2. Overusing IServiceProvider
3. Not using interfaces
4. Too many dependencies in constructor
5. Ignoring lifetimes

Need Deeper Guidance?
If you want real 1-on-1 help:
Complete Interview Q&A Resources, Live mock interviews,Career mentorship,Resume reviews,LinkedIn profile optimization Connect with me:

https://topmate.io/sandeeppal

https://www.linkedin.com/in/sandeeppal/